Vision of the Department

To be a well-known centre for pursuing computer education through innovative pedagogy, value-based education and industry collaboration.

Mission of the Department

To establish learning ambience for ushering in computer engineering professionals in core and multidisciplinary area by developing Problem-solving skills through emerging technologies.

Session 2025-2026

Vision: Dream of where you want.	**Mission:** Means to achieve Vision

Program Educational Objectives of the program (PEO): (broad statements that describe the professional and career accomplishments)

PEO1	**Preparation**	**P: Preparation**	**Pep-CL abbreviation pronounce as Pep-si-lL easy to recall**
PEO2	**Core Competence**	**E: Environment (Learning Environment)**	
PEO3	**Breadth**	**P: Professionalism**	
PEO4	**Professionalism**	**C: Core Competence**	
PEO5	**Learning Environment**	**L: Breadth (Learning in diverse areas)**	

Program Outcomes (PO): (statements that describe what a student should be able to do and know by the end of a program)

Keywords of POs:
Engineering knowledge, Problem analysis, Design/development of solutions, Conduct Investigations of Complex Problems, Engineering Tool Usage, The Engineer and The World, Ethics, Individual and Collaborative Team work, Communication, Project Management and Finance, Life-Long Learning

PSO Keywords: Cutting edge technologies, Research

"I am an engineer, and I know how to apply engineering knowledge to investigate, analyse and design solutions to complex problems using tools for entire world following all ethics in a collaborative way with proper management skills throughout my life." *to contribute to the development of cutting-edge technologies and Research*.

Integrity: I will adhere to the Laboratory Code of Conduct and ethics in its entirety.

Name and Signature of Student and Date
(Signature and Date in Handwritten)

Vision of the Department

To be a well-known centre for pursuing computer education through innovative pedagogy, value-based education and industry collaboration.

Mission of the Department

To establish learning ambience for ushering in computer engineering professionals in core and multidisciplinary area by developing Problem-solving skills through emerging technologies.

Session	**2025-26 (ODD)**	**Course Name**	**Operating System Lab**
Semester	**5**	**Course Code**	**23IOT1504**
Roll No	**64**	**Name of Student**	**Sheikh Wasimuddin**

Practical Number	**09**
Course Outcome	1. Understand Computer System Configuration and Simulate system resources efficiently using Linux Commands (CO1) 2. Analyse operating system functionalities utilizing system calls, thread programming and process scheduling algorithms (CO2) 3. Apply Synchronization primitives to implement a Deadlock-free solution(CO3) 4. Simulate Disk scheduling, Memory allocation, File allocation, page replacement algorithms (CO4)
Aim	Simulate all file allocation strategies A. Sequential B. Indexed C. Linked
Theory	File allocation methods determine how disk blocks are assigned to files in a file system. The three main strategies are Sequential (Contiguous), Linked, and Indexed allocation. In Sequential allocation, every file is stored in continuous blocks on the disk. The directory holds the starting address and the length of the file. This method is simple and gives fast access for both sequential and random access operations. However, it suffers from external fragmentation and requires knowing the file size in advance, making file growth difficult. In Linked allocation, each file is stored in blocks scattered anywhere on the disk. Every block contains a pointer to the next block, forming a chain. This eliminates external fragmentation and allows files to grow easily, but random access becomes slow and there is overhead of storing pointers. Also, if a pointer is lost or damaged, the file may become corrupted. In Indexed allocation, a special index block is used for each file, containing the addresses of all disk blocks assigned to that file. This method supports direct and sequential access efficiently and avoids

	external fragmentation. The drawback is that storing an index block takes extra space and may become large for big files. These allocation strategies offer different trade-offs in terms of performance, memory usage, and file growth flexibility, allowing operating systems to choose based on specific requirements.

Procedure and Execution	**A. Sequential**
	Code:
	// sheikh wasimuddin 23071044 CSEIOT_64
	#include <stdio.h>
	int main() {
	int files[50], start, length, i, j;
	for (i = 0; i < 50; i++)
	files[i] = 0;
	printf("Enter starting block and length of file: ");
	scanf("%d %d", &start, &length);
	for (i = start; i < (start + length); i++) {
	if (files[i] == 1) {
	printf("Block %d is already allocated!\n", i);
	return 0;
	}
	}
	for (i = start; i < (start + length); i++)
	files[i] = 1;
	printf("\nFile allocated from block %d to %d.\n", start, start + length - 1);
	printf("Blocks allocated: ");
	for (i = start; i < (start + length); i++)
	printf("%d ", i);
	printf("\n");
	return 0;
	}
	
	B. Indexed
	Code
	// sheikh wasimuddin 23071044 CSEIOT_64
	#include <stdio.h>
	int main() {
	int files[50], indexBlock, i, n, blocks[50];
	for (i = 0; i < 50; i++)
	files[i] = 0;
	printf("Enter index block: ");
	scanf("%d", &indexBlock);
	if (files[indexBlock] == 1) {
	printf("Index block already allocated!\n");

```
return 0;
}
files[indexBlock] = 1;
printf("Enter number of blocks to be allocated: ");
scanf("%d", &n);
printf("Enter block numbers: ");
for (i = 0; i < n; i++)
scanf("%d", &blocks[i]);
for (i = 0; i < n; i++) {
if (files[blocks[i]] == 1) {
printf("Block %d already allocated!\n", blocks[i]);
return 0;
}
}
for (i = 0; i < n; i++)
files[blocks[i]] = 1;
printf("\nFile allocated with index block %d\n", indexBlock);
printf("Index block %d points to: ", indexBlock);
for (i = 0; i < n; i++)
printf("%d ", blocks[i]);
printf("\n");
return 0;
}
```

C. Linked
Code

```
// sheikh wasimuddin 23071044 CSEIOT_64
#include <stdio.h>
#include <stdlib.h>
struct Node {
 int block;
 struct Node *next;
};
int main() {
 int files[50], n, i, blocks[50];
 struct Node *head = NULL, *temp = NULL, *newNode = NULL;
 for (i = 0; i < 50; i++)
 files[i] = 0;
 printf("Enter number of blocks to allocate: ");
 scanf("%d", &n);
 printf("Enter block numbers: ");
 for (i = 0; i < n; i++)
 scanf("%d", &blocks[i]);
 for (i = 0; i < n; i++) {
```

```
if (files[blocks[i]] == 1) {
printf("Block %d is already allocated!\n", blocks[i]);
return 0;
}
}
for (i = 0; i < n; i++) {
files[blocks[i]] = 1;
newNode = (struct Node*)malloc(sizeof(struct Node));
newNode->block = blocks[i];
newNode->next = NULL;
if (head == NULL)
head = newNode;
else
temp->next = newNode;
temp = newNode;
}
printf("\nLinked allocation sequence:\n");
temp = head;
while (temp != NULL) {
printf("%d -> ", temp->block);
temp = temp->next;
}
printf("NULL\n");
return 0;
}
```

Output:

A. Sequential

main.c

```
// sheikh wasimuddin 23071044 CSEIOT_64
#include <stdio.h>
int main() {
 int files[50], start, length, i, j;
 for (i = 0; i < 50; i++)
 files[i] = 0;
 printf("Enter starting block and length of file: ");
 scanf("%d %d", &start, &length);
 for (i = start; i < (start + length); i++) {
 if (files[i] == 1) {
 printf("Block %d is already allocated!\n", i);
 return 0;
 }
 }
 for (i = start; i < (start + length); i++)
 files[i] = 1;
 printf("\nFile allocated from block %d to %d.\n", start, start + length - 1);
 printf("Blocks allocated: ");
 for (i = start; i < (start + length); i++)
 printf("%d ", i);
 printf("\n");
 return 0;
}
```

```
Enter starting block and length of file: 7 4

File allocated from block 7 to 10.
Blocks allocated: 7 8 9 10


...Program finished with exit code 0
Press ENTER to exit console.
```

B. Indexed

main.c

```c
// sheikh wasimuddin 23071044 CSEIOT_64
#include <stdio.h>
int main() {
 int files[50], indexBlock, i, n, blocks[50];
 for (i = 0; i < 50; i++)
 files[i] = 0;
 printf("Enter index block: ");
 scanf("%d", &indexBlock);
 if (files[indexBlock] == 1) {
 printf("Index block already allocated!\n");
 return 0;
 }
 files[indexBlock] = 1;
 printf("Enter number of blocks to be allocated: ");
 scanf("%d", &n);
 printf("Enter block numbers: ");
 for (i = 0; i < n; i++)
 scanf("%d", &blocks[i]);
 for (i = 0; i < n; i++) {
 if (files[blocks[i]] == 1) {
 printf("Block %d already allocated!\n", blocks[i]);
 return 0;
 }
 }
 for (i = 0; i < n; i++)
 files[blocks[i]] = 1;
 printf("\nFile allocated with index block %d\n", indexBlock);
 printf("Index block %d points to: ", indexBlock);
 for (i = 0; i < n; i++)
 printf("%d ", blocks[i]);
 printf("\n");
 return 0;
}
```

```
Enter index block: 6
Enter number of blocks to be allocated: 4
Enter block numbers: 8 10 12 14

File allocated with index block 6
Index block 6 points to: 8 10 12 14


...Program finished with exit code 0
Press ENTER to exit console.
```

C. Linked

main.c

```c
// sheikh wasimuddin 23071044 CSEIOT_64
#include <stdio.h>
#include <stdlib.h>
struct Node {
 int block;
 struct Node *next;
};
int main() {
 int files[50], n, i, blocks[50];
 struct Node *head = NULL, *temp = NULL, *newNode = NULL;
 for (i = 0; i < 50; i++)
 files[i] = 0;
 printf("Enter number of blocks to allocate: ");
 scanf("%d", &n);
 printf("Enter block numbers: ");
 for (i = 0; i < n; i++)
 scanf("%d", &blocks[i]);
 for (i = 0; i < n; i++) {
 if (files[blocks[i]] == 1) {
 printf("Block %d is already allocated!\n", blocks[i]);
 return 0;
 }
 }
```

```
Enter number of blocks to allocate: 5
Enter block numbers: 7 10 8 18 21

Linked allocation sequence:
7 -> 10 -> 8 -> 18 -> 21 -> NULL


...Program finished with exit code 0
Press ENTER to exit console.
```

Output Analysis	The three allocation strategies present a fundamental trade-off between efficiency and flexibility. Sequential allocation is the simplest and fastest for sequential reading, as all file blocks are contiguous, but it suffers critically from external fragmentation, where free space is broken into small, unusable chunks, making it difficult to allocate new or grow existing files. Linked allocation solves external fragmentation completely by linking scattered blocks, allowing files to grow easily, but it pays a heavy price in performance: sequential access is slow due to constant disk seeks, and direct access is impossible as you must traverse the chain. Indexed allocation provides a robust compromise by using an index block to store pointers to all data blocks; this eliminates external fragmentation while

	also providing fast direct access (unlike linked). Its main drawback is the overhead of the index block itself, which wastes space, especially for many small files
Conclusion	In conclusion, no single file allocation strategy is universally superior; each represents a distinct trade-off. Sequential allocation offers the best performance for sequential reads but is inflexible and suffers from external fragmentation. Linked allocation completely solves fragmentation and allows easy file growth, but it fails to provide efficient direct access and is slow for sequential reads. Indexed allocation emerges as a balanced, modern solution that defeats external fragmentation (like linked) while providing fast direct access (unlike linked), making it highly flexible. Its only significant drawback is the storage overhead required for the index blocks themselves, a cost that is generally considered acceptable in modern systems for the powerful flexibility it provides.
Plag Report (Similarity index < 12%)	
Date	03/11/2025